Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Праграмаванне на мовах высокага ўзроўню

К ЗАЩИТЕ ДОПУСТИТЬ

_____ Кавальчук А. М.

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему

ІНФАРМАЦЫЙНАЯ СІСТЭМА ПА ПРОДАЖЫ КАМП'ЮТАРНЫХ КАМПЛЕКТУЮЧЫХ

БДУІР КР 6-05-0611-05 316 ПЗ

Студент Мінкевіч А. С.
(гр. 550503)

Руководитель Кавальчук А. М.

МІНСК 2026

ЗМЕСТ

УВОДЗІНЫ 3
1. АГЛЯД ЛІТАРАТУРЫ 5
   1.1. Агляд аналагаў 5
   1.2. Агляд інфармацыйных тэхналогій 6
   1.3. Пастаноўка задачы 7
2. ФУНКЦЫЯНАЛЬНАЕ ПРАЕКТАВАННЕ 8
   2.1. Структура ўваходных і выходных дадзеных 8
   2.2. Распрацоўка іерархіі класаў і архітэктуры 11
   2.3. Апісанне асноўных класаў і кантэйнераў 11
   2.4. Алгарытмы асноўных функцый 13
3. РАСПРАЦОЎКА ПРАГРАМНЫХ МОДУЛЯЎ 15
4. РЭЗУЛЬТАТЫ РАБОТЫ І ТЭСЦІРАВАННЕ 17
5. КІРАЎНІЦТВА КАРЫСТАЛЬНІКА 18
ЗАКЛЮЧЭННЕ 19
СПІС ВЫКАРЫСТАНЫХ КРЫНІЦ 20

УВОДЗІНЫ

У сучасным свеце інфармацыйныя тэхналогіі імкліва развіваюцца, і вылічальная тэхніка стала неад'емнай часткай штодзённага жыцця людзей, а таксама асновай функцыянавання любога бізнесу. Гэта спараджае стабільна высокі попыт на персанальныя камп'ютары, сервернае абсталяванне і, адпаведна, на асобныя камп'ютарныя камплектуючыя: працэсары, відэакарты, мацярынскія платы, аператыўную памяць і назапашвальнікі дадзеных.

Рынак камп'ютарных кампанентаў адрозніваецца надзвычайнай дынамічнасцю: новыя пакаленні прылад з'яўляюцца штогод, а асартымент сярэдняй крамы налічвае тысячы найменняў. Пры гэтым кожная дэталь мае мноства спецыфічных тэхнічных характарыстык (тыпы сокетаў, частата, аб'ём памяці, форма-фактар, энергаспажыванне і інш.). Улічваць такі складаны, шматкампанентны і разнастайны тавар уручную або з дапамогай звычайных электронных табліц вельмі складана. Чалавечы фактар пры ручным уліку непазбежна прыводзіць да памылак: узнікае блытаніна ў наменклатуры, губляецца актуальная інфармацыя аб наяўнасці тавару на складзе, што ў выніку вядзе да зрыву заказаў і зніжэння якасці абслугоўвання пакупнікоў.

Менавіта таму *актуальнасць тэмы* дадзенай курсавой працы з'яўляецца бясспрэчнай. Аўтаматызацыя працэсу продажу і ўліку камп'ютарных камплектуючых шляхам укаранення надзейнай лакальнай інфармацыйнай сістэмы — гэта крытычна важная задача для любой сучаснай крамы тэхнікі. Спецыялізаваная сістэма дазваляе дакладна структураваць каталог тавараў, забяспечыць імгненны пошук і фільтрацыю па зададзеных крытэрыях, кантраляваць складскія рэшткі і пазбягаць памылак пры афармленні заказаў.

Мэтай дадзенай курсавой працы з'яўляецца праектаванне і распрацоўка інфармацыйнай сістэмы па продажы камп'ютарных камплектуючых з выкарыстаннем аб'ектна-арыентаванага падыходу на мове праграмавання C++.

Для дасягнення пастаўленай мэты неабходна вырашыць наступныя задачы:

- правесці аналіз прадметнай вобласці (спецыфікі продажу камплектуючых) і вызначыць патрабаванні да праграмы;
- выканаць агляд існуючых сістэм-аналагаў і абгрунтаваць выбар інструментальных сродкаў распрацоўкі;
- спраектаваць архітэктуру праграмы, стварыўшы іерархію класаў для прадстаўлення розных тыпаў абсталявання;

– рэалізаваць уласны шаблонны кантэйнер з ітэратарам і ўласныя алгарытмы, а таксама выкарыстаць сродкі STL;
– распрацаваць функцыі для дадання, рэдагавання, выдалення, пошуку і сартавання тавараў у каталогу;
– рэалізаваць адмену апошніх дзеянняў, апрацоўку выключных сітуацый і журналіраванне аперацый;
– правесці тэсціраванне праграмы і апісаць вынікі.

1. АГЛЯД ЛІТАРАТУРЫ

1.1. Агляд аналагаў

На сучасным рынку ёсць шмат праграм для кіравання крамамі і складамі, накшталт WMS і ERP [1]. Каб зразумець, што трэба зрабіць і чаго чакаць ад новай сістэмы, разгледзім тыя, што ўжо існуюць.

1. «1С:Прадпрыемства» (з канфігурацыямі «Кіраванне гандлем» або «1С:Склад»). Гэтая праграма вельмі папулярная ў краінах СНД [2]. Яна ўмее ўлічваць тавары, працаваць з пастаўшчыкамі, весці ўлік па партыях (FIFO, LIFO, сярэдні сабекошт), выкарыстоўваць сканеры штрыхкодаў і фіскальныя рэгістратары. Плюсы: добра прадумана з пункту гледжання бізнесу, адпавядае заканадаўству і бухгалтарскім нормам, добра маштабуецца. Мінусы: дарагія ліцэнзіі і абслугоўванне; інтэрфейс занадта складаны для маленькіх крам; патрабуе магутнага сервера і баз дадзеных накшталт MS SQL Server або PostgreSQL; платформу нельга мяняць, яна працуе толькі ўнутры «1С».
2. «МойСклад». Гэта воблачны сэрвіс, які падыходзіць малому і сярэдняму бізнесу. Ён дазваляе весці ўлік рэшткаў, запісваць пакупкі і продажы праз інтэрнэт. Плюсы: зручны і сучасны інтэрфейс, доступ да сістэмы з любога месца без усталёўкі, аўтаматычнае рэзервовае капіраванне дадзеных у воблаку. Мінусы: без інтэрнэту працаваць нельга; трэба плаціць кожны месяц; нельга наладзіць пад аўтаномныя сканеры дадзеных; захоўванне дадзеных у трэціх асоб можа быць рызыкоўным.
3. Odoo Warehouse. Гэта модуль з ERP-сістэмы з адкрытым кодам. Ён дапамагае кіраваць складамі, перамяшчэннем тавараў і адрасным захоўваннем. Плюсы: складаецца з асобных блокаў, адкрыты код, само ядро сістэмы бясплатнае. Мінусы: цяжка ўсталяваць і наладзіць (патрэбныя Linux, Python, PostgreSQL); моцна нагружае камп’ютар нават для простых аперацый; не падыходзіць для працы без сеткі.

Праведзены аналіз паказвае, што для лакальных тэрміналаў і аўтаномных філіялаў мэтазгодна распрацоўка кампактнага праграмнага прадукту, які не патрабуе разгортвання цяжкавагавых СУБД і сеткавых службаў. Праектуемая сістэма павінна спалучаць высокую прадукцыйнасць апрацоўкі дадзеных,

дэтэрмінаваную працу з лакальнымі файламі і інтуітыўна зразумелы інтэрфейс.

1.2. Агляд інфармацыйных тэхналогій

Пры прыняцці рашэння аб тэхналагічным інструментарыі для рэалізацыі сістэмы ўліку асабліва важнымі фактарамі з'яўляюцца: хуткадзейнасць алгарытмаў апрацоўкі таблічных структур, бяспека кіравання памяццю, магчымасць падтрымкі аб'ектна-арыентаванай дэкампазіцыі і незалежнасць ад знешніх старонніх сэрвісаў.

У ходзе папярэдняга аналізу ў якасці разглядаемых моў былі выбраны мовы C#, Python і C++. Мова Python дазваляе хутка пісаць праграмны код, аднак дынамічная тыпізацыя і інтэрпрэтуемы характар выканання азначаюць невысокую хуткасць працы з вялікімі аб'ёмамі наменклатурных масіваў. Зборшчык смецця (Garbage Collector) не забяспечвае своечасовае вызваленне сістэмных файлавых дэскрыптараў аперацыйнай сістэмы [3].

Мова C# (платформа .NET) прапануе развітыя сродкі пабудовы бізнес-дадаткаў, але патрабуе абавязковага асяроддзя выканання CLR, што павялічвае час халоднага старту і спажыванне аператыўнай памяці.

Найбольш прыдатным для пастаўленых задач інструментам выбрана мова праграмавання C++ (стандарт C++17) [4]. Абгрунтаванне выбару заснавана на наступным:

– кампіляцыя ў машынны код і прамое кіраванне памяццю: забяспечвае максімальна магчымую хуткадзейнасць пры правядзенні разлікова-аналітычных аперацый і мінімальныя апаратныя патрабаванні да тэрміналаў [5];
– парадыгма RAII: захоп рэсурсаў аўтаматычна звязваецца з часам жыцця аб'екта, а разумныя ўказальнікі `std::unique_ptr` выключаюць уцечкі памяці пры ўзнікненні няштатных сітуацый [6];
– развітыя механізмы аб'ектна-арыентаванага праграмавання: дазваляюць фармалізаваць прадметную вобласць у выглядзе іерархіі класаў, раз'ядноўваючы інтэрфейс і ўнутраную імплементацыю [7];
– абагульненае праграмаванне (шаблоны): дазваляе без страты тыпабяспекі рэалізаваць уласныя кантэйнеры, ітэратары і алгарытмы [8];

– развітыя сродкі ўводу-вываду і маніпуляцыі кансольным асяроддзем: дазваляюць стварыць эрганамічны дыялогавы інтэрфейс аператара з абаронай ад некарэктных дадзеных.

Рашэнне па інтэграваным асяроддзі распрацоўкі (IDE) пераважна спыніць на асяроддзі Visual Studio Code (VS Code) [9]. Кампілятарам у сістэме выкарыстоўваецца GCC (у зборцы MinGW-w64), збіранне праекта выконваецца праз CMake. Захоўванне дадзеных мяркуецца ў шматфайлавай структуры з выкарыстаннем тэкставых файлаў для кожнай асобнай катэгорыі камплектуючых.

1.3. Пастаноўка задачы

Праграма павінна мець зручны карыстальніцкі інтэрфейс з неабходнымі пунктамі меню. Уся інфармацыя павінна знаходзіцца ў розных тэкставых файлах, звязаных па катэгорыях і ідэнтыфікатарах.

Рэалізаваць наступныя функцыі: даданне, рэдагаванне, выдаленне дадзеных па камплектуючых, пошук дадзеных па розных прыкметах, сартаванне, фарміраванне справаздачы па складскіх рэштках і журналіраванне. Дадаткова рэалізуецца падзел правоў доступу: адміністратар можа змяняць каталог, карыстальнік — толькі праглядаць, шукаць, сартаваць і фарміраваць справаздачу.

Распрацаваць іерархію класаў з выкарыстаннем наследавання (не менш за 3 ўзроўні наследавання). Распрацаваць і выкарыстоўваць класы кантэйнераў, ітэратараў і алгарытмаў (свае і STL). Праводзіць апрацоўку выключных сітуацый. Пры рэалізацыі аперацый рэдагавання, дадання і выдалення інфармацыі неабходна прадугледзець аперацыю адмены апошніх дзеянняў.

2. ФУНКЦЫЯНАЛЬНАЕ ПРАЕКТАВАННЕ

2.1. Структура ўваходных і выходных дадзеных

У дадзеным падраздзеле вызначаюцца крыніцы, фарматы і абмежаванні дадзеных, якія выкарыстоўваюцца ў сістэме на этапах уводу, апрацоўкі і доўгатэрміновага захоўвання.

Уваходныя дадзеныя праграмы фарміруюцца з двух крыніц:

1. Карыстальніцкі ўвод з клавіятуры:
   - выбар ролі і пунктаў дыялогавага меню: цэлыя лікі ў дыяпазоне ад `MENU_EXIT` да `MENU_MAX`;
   - радковыя параметры тавару: артыкул (да `ARTICLE_LEN` сімвалаў), найменне і вытворца (да `NAME_LEN` сімвалаў); сімвал «;» у радках забаронены, бо выкарыстоўваецца як раздзяляльнік палёў у файлах;
   - лічбавыя параметры тавару: кошт у дыяпазоне ад `MIN_PRICE` да `MAX_PRICE`; колькасць на складзе ад `MIN_QUANTITY` да `MAX_QUANTITY` адзінак;
   - тэхнічныя характарыстыкі, якія залежаць ад тыпу камплектуючага: тып падключэння (USB, Wireless, HDMI, DisplayPort) для перыферыі, DPI для мышэй, тып пераключальнікаў для клавіятур, частата абнаўлення для маніторaў, энергаспажыванне TDP для ўнутраных камплектуючых, тактавая частата для працэсараў, аб’ём для аператыўнай памяці.
2. Файлавы ўвод:
   - файл `categories.txt`: спіс катэгорый тавараў і назвы звязаных з імі файлаў захоўвання;
   - файлы катэгорый: `mice.txt`, `keyboards.txt`, `monitors.txt`, `processors.txt`, `ram.txt`.

Поля ў радку файла раздзяляюцца сімвалам «;». Пры загрузцы кожны радок правяраецца: радкі з няпоўным наборам палёў, недапушчальнымі значэннямі або паўторным артыкулам прапускаюцца, а іх колькасць паведамляецца карыстальніку.

Выходнымі дадзенымі сістэмы з’яўляюцца:

– экранны вывад: фарматаванае прадстаўленне каталога камплектуючых у выглядзе табліц з выраўнаванымі слупкамі (асобная табліца для кожнай катэгорыі);
– файлы катэгорый: захаванне абноўленай інфармацыі пры змяненні, даданні ці выдаленні запісаў;
– файл `inventory_report.txt`: тэкставая табліца са справаздачай па складскіх рэштках і разлікам агульнага кошту тавараў;
– файл `logs.txt`: даданне запісаў з фіксацыяй часу, ролі карыстальніка і выкананага дзеяння.

Ніжэй прыведзены макеты і прыклады напаўнення файлаў інфармацыйнай сістэмы.

Табліца 2.1 — Структура і прыклад напаўнення файла categories.txt

ID	Назва катэгорыі	Файл захоўвання	Апісанне
1	Мышы	mice.txt	Правадныя і бесправадныя мышы
2	Клавіятуры	keyboards.txt	Механічныя і мембранныя клавіятуры
3	Маніторы	monitors.txt	Дысплеі розных дыяганаляў і частот
4	Працэсары	processors.txt	Цэнтральныя працэсары (CPU) для ПК
5	Аператыўная памяць	ram.txt	Модулі аператыўнай памяці

Табліца 2.2 — Структура і прыклад напаўнення файла mice.txt

ID	Артыкул	Найменне	Вытворца	Кошт	Колькасць	Падключэнне	DPI
101	M-001	G Pro Wireless	Logitech	350.50	15	Wireless	25600
102	M-002	DeathAdder V2	Razer	180.00	22	USB	20000
103	M-003	Viper Mini	Razer	110.00	30	USB	8500

Табліца 2.3 — Структура і прыклад напаўнення файла keyboards.txt

ID	Артыкул	Найменне	Вытворца	Кошт	Колькасць	Падключэнне	Тып пераключ.
201	K-012	Apex Pro TKL	SteelSeries	600.00	8	USB	Mechanical
202	K-013	Alloy Origins	HyperX	310.00	14	USB	Mechanical
203	K-014	K120	Logitech	45.00	50	USB	Membrane

Табліца 2.4 — Структура і прыклад напаўнення файла monitors.txt

ID	Артыкул	Найменне	Вытворца	Кошт	Колькасць	Падключэнне	Частата (Гц)
301	MO-05	UltraGear 27	LG	950.00	5	HDMI	144
302	MO-06	TUF Gaming VG249	ASUS	620.00	11	DisplayPort	165
303	MO-07	Odyssey G5	Samsung	830.00	7	HDMI	144

Табліца 2.5 — Структура і прыклад напаўнення файла processors.txt

ID	Артыкул	Найменне	Вытворца	Кошт	Колькасць	TDP (Вт)	Частата (МГц)
401	CPU-9	Ryzen 5 5600X	AMD	550.00	20	65	3700
402	CPU-10	Core i5-12400F	Intel	490.00	18	65	2500
403	CPU-11	Ryzen 7 5800X3D	AMD	1050.00	6	105	3400

Табліца 2.6 — Структура і прыклад напаўнення файла ram.txt

ID	Артыкул	Найменне	Вытворца	Кошт	Колькасць	TDP (Вт)	Аб'ём (ГБ)
501	RAM-1	Vengeance LPX 16GB DDR4	Corsair	120.00	25	5	16
502	RAM-2	Fury Beast 32GB DDR5	Kingston	260.00	12	6	32

Табліца 2.7 — Структура і прыклад напаўнення выходнага файла inventory_report.txt

№	Артыкул	Найменне тавару	Вытворца	Колькасць	Агульны кошт
1	M-001	G Pro Wireless	Logitech	15	5257.50
2	K-012	Apex Pro TKL	SteelSeries	8	4800.00
3	MO-05	UltraGear 27	LG	5	4750.00
4	CPU-9	Ryzen 5 5600X	AMD	20	11000.00

Табліца 2.8 — Структура і прыклад напаўнення файла часопіса logs.txt

Дата і час	Роля	Выкананае дзеянне
07.11.2026 18:30	Сістэма	Загрузка дадзеных з файлаў катэгорый
07.11.2026 18:30	Адміністратар	Даданне RAM-9
07.11.2026 18:31	Адміністратар	Адмена: Даданне RAM-9
07.11.2026 18:32	Карыстальнік	Фарміраванне справаздачы inventory_report.txt

2.2. Распрацоўка іерархіі класаў і архітэктуры

У адпаведнасці з патрабаваннямі да аб’ектна-арыентаванай мадэлі ў праграме распрацавана іерархія класаў, якая складаецца з трох узроўняў наследавання.

Іерархія класаў прадстаўлена наступным чынам:

1. **Першы ўзровень (базавы абстрактны клас):** `Component` — утрымлівае агульныя ўласцівасці любога тавару: ID, артыкул, найменне, вытворцу, кошт і колькасць.
2. **Другі ўзровень (абстрактныя прамежкавыя класы):**
   - `PeripheralDevice` (перыферыйныя прылады) — дадае тып падключэння;
   - `InternalHardware` (унутраныя камплектуючыя) — дадае энергаспажыванне TDP.
3. **Трэці ўзровень (канкрэтныя класы):**
   - `Mouse` (DPI), `Keyboard` (тып пераключальнікаў) і `Monitor` (частата абнаўлення) наследуюцца ад `PeripheralDevice`;
   - `Processor` (тактавая частата) і `RAM` (аб’ём) наследуюцца ад `InternalHardware`.

Кожны клас сам запісвае і счытвае ўласныя палі праз віртуальныя метады `serialize()` і `deserialize()`: метад нашчадка спачатку выклікае метад бацькоўскага класа, а затым апрацоўвае дададзеныя палі. Пры загрузцы файла аб’ект ствараецца фабрыкай `ComponentFactory::create()` па ідэнтыфікатары катэгорыі і захоўваецца ў кантэйнеры праз `std::unique_ptr<Component>`, што забяспечвае палімарфізм і аўтаматычнае вызваленне памяці.

2.3. Апісанне асноўных класаў і кантэйнераў

Клас `Component` (абстрактны):

- **Палі (protected):** `id_`, `article_`, `name_`, `manufacturer_`, `price_`, `quantity_`.
- **Віртуальныя метады:** `category_id()`, `field_names()`, `set_field()`, `serialize()`, `deserialize()`, `print_header()`, `print_row()`, `clone()`; дэструктар віртуальны.

– **Валідацыя:** сэтэры `set_price()`, `set_quantity()`, `set_article()` і іншыя правяраюць значэнні і пры памылцы выкідваюць `ValidationException`, пакідаючы аб’ект ранейшым.

– **Прызначэнне:** выкарыстоўваецца для стварэння ўказальнікаў базавага тыпу пры рэалізацыі палімарфізму і захоўванні розных тавараў у адным кантэйнеры.

Класы `Mouse`, `Keyboard`, `Monitor`, `Processor`, `RAM`: перавызначаюць віртуальныя метады бацькоўскіх класаў (`override`), дадаюць унікальныя палі і сэтэры з праверкай дыяпазону (напрыклад, `set_dpi()`). Метад `clone()` стварае глыбокую копію аб’екта і выкарыстоўваецца для захавання стану перад рэдагаваннем. Метад `set_field(номер, значэнне)` дазваляе змяніць поле па нумары, не звязваючы мадэль з кансольным вводам.

Шаблон `VectorContainer<T>`:

– Уласны дынамічны масіў. Капіраванне забаронена, дазволена толькі перамяшчэнне, таму ў ім можна захоўваць `std::unique_ptr`.

– **Метады:** `push_back()`, `insert()`, `erase()` (вяртае выдалены элемент), `operator[]` з праверкай межаў, `size()`, `begin()`, `end()`.

– **Унутраны шаблон ітэратара `BasicIterator`:** перагружаныя аператары `++`, `*`, `->`, `==`, `!=`; ёсць варыянты для звычайнага і канстантнага доступу.

Уласныя алгарытмы (`Algorithms.h`): `find_if_own()`, `count_if_own()` і `sort_own()` (сартаванне выбарам). Яны патрабуюць ад ітэратара толькі аперацыі `++`, `*`, `!=` і капіявання, таму працуюць і з уласным кантэйнерам, і з `std::vector`. З бібліятэкі STL выкарыстоўваюцца `std::stack`, `std::vector`, `std::map`, `std::sort`, `std::accumulate`, `std::find`.

Іерархія выключэнняў: `InventoryException` (наследуецца ад `std::runtime_error`) і вытворныя `ValidationException` (недапушчальныя значэнні), `FileException` (памылкі файлаў і фармату), `NotFoundException` (тавар або катэгорыя не знойдзены).

Механізм адмены (Undo): рэалізаваны па шаблоне «Каманда». Абстрактны клас `Command` мае метады `execute()`, `undo()` і `description()`. Канкрэтныя каманды захоўваюць даныя для адкату:

– `AddCommand` — пры адмене вяртае дададзены тавар з кантэйнера;

– `RemoveCommand` — запамінае індэкс і сам аб’ект, пры адмене вяртае яго на ранейшае месца;

– `EditCommand` — перад зменай захоўвае копію аб’екта праз `clone()`, пры адмене падстаўляе яе назад.

Каманды змяшчаюцца ў `std::stack`, таму адмена выконваецца ў парадку LIFO, і індэксы астаюцца карэктнымі. Сартаванне не змяняе парадак у кантэйнеры (сартуецца копія спісу ўказальнікаў), каб не парушыць індэксы ў камандах.

Клас `InventoryManager`: кіруе каталогам і інкапсулюе кантэйнер, спіс катэгорый і стэк каманд. Метады: `load_all()`, `save_all()`, `add()`, `remove()`, `edit()`, `undo()`, `find_by_article()`, `search_by_manufacturer()`, `search_by_name()`, `search_by_price()`, `sorted()`, `write_report()`. Метад `save_all()` запісвае кожны аб’ект у файл сваёй катэгорыі, назва якога чытаецца з `categories.txt`.

Класы `Logger` і `ComponentFactory`: `Logger::log()` дапісвае ў `logs.txt` дату, ролю і дзеянне; `ComponentFactory` стварае аб’ект па ID катэгорыі.

2.4. Алгарытмы асноўных функцый

Загрузка дадзеных (`load_all()`). Спачатку чытаецца `categories.txt`; затым для кожнай катэгорыі адкрываецца яе файл. Для кожнага непустога радка фабрыка стварае аб’ект, ён чытае свае палі, пасля чаго правяраецца адсутнасць лішніх палёў і паўтору артыкула. Любая `InventoryException` лічыцца памылкай толькі гэтага радка: радок прапускаецца, паведамленне з нумарам радка выводзіцца на экран, а загрузка працягваецца.

Даданне. Карыстальнік выбірае катэгорыю, фабрыка стварае пусты аб’ект, прысвойваецца наступны вольны ID (`next_id()`), уводзіцца артыкул і значэнні палёў у цыкле па `field_names()`; пры памылцы валідацыі значэнне запытваецца паўторна. Гатовы аб’ект перадаецца ў `AddCommand`, пасля чаго каманда выконваецца і кладзецца ў стэк.

Рэдагаванне і выдаленне. Тавар шукаецца па артыкуле (`NotFoundException`, калі яго няма). Для рэдагавання каманда спачатку змяняе поле і толькі пры паспяховай валідацыі трапляе ў стэк, таму няўдалае рэдагаванне не пакідае след у гісторыі.

Пошук і сартаванне. Пошук выконваецца па артыкуле, часткі найменання, вытворцы (без уліку рэгістра для лацінскіх літар) і дыяпазоне кошту. Сартаванне магчыма па кошце, назве і колькасці як уласным алгарытмам, так і `std::sort`.

Справаздача. `write_report()` запісвае для кожнага тавару артыкул, найменне, вытворцу, колькасць і агульны кошт (кошт на колькасць), а ў канцы — колькасць пазіцый і агульную суму, падлічаную праз `std::accumulate`.

3. РАСПРАЦОЎКА ПРАГРАМНЫХ МОДУЛЯЎ

Праграма складаецца з наступных модуляў (кожны модуль — пара файлаў `.h` / `.cpp`, шаблоны размешчаны цалкам у загалоўках):

- `constants.h`, `exceptions.h`, `validation.h` — канстанты, выключэнні і функцыі праверкі і пераўтварэння ўваходных радкоў (`check_text()`, `check_range()`, `parse_int()`, `parse_double()`);
- `Component`, `Peripherals`, `Internals` — іерархія класаў тавараў;
- `ComponentFactory` — стварэнне аб’ектаў па ідэнтыфікатары катэгорыі;
- `VectorContainer.h`, `Algorithms.h` — уласны кантэйнер, ітэратар і алгарытмы;
- `Command.h` — каманды адмены;
- `InventoryManager` — бізнес-логіка і праца з файламі;
- `Logger` — журналіраванне;
- `main.cpp` — дыялогавае меню і абмежаванне правоў па ролях.

Ніжэй прыведзены ключавыя фрагменты. Метад `erase()` кантэйнера вяртае выдалены элемент, што дазваляе камандзе выдалення вярнуць яго пры адмене:

```
T erase(size_t index)
{
    if (index >= size_)
        throw std::out_of_range("VectorContainer::erase: индекс вне
диапазона");
    T removed = std::move(data_[index]);
    for (size_t i = index; i + 1 < size_; ++i)
        data_[i] = std::move(data_[i + 1]);
    data_[size_ - 1] = T();
    --size_;
    return removed;
}
```

Каманда рэдагавання захоўвае копію аб’екта і змяняе поле толькі пасля паспяховай валідацыі:

```
void EditCommand::execute()
{
    auto backup = items_[index_]->clone();
    items_[index_]->set_field(field_, value_);
    old_ = std::move(backup);
}
void EditCommand::undo() { items_[index_] = std::move(old_); }
```

Сартаванне выбарам працуе праз любы ітэратар з аперацыямі ++, *, !=:

```
template <typename It, typename Less>
void sort_own(It first, It last, Less less)
{
    for (It i = first; i != last; ++i)
    {
        It smallest = i;
        It j = i;
        ++j;
        for (; j != last; ++j)
            if (less(*j, *smallest))
                smallest = j;
        if (smallest != i)
            std::swap(*i, *smallest);
    }
}
```

Серыялізацыя працуе ланцужком віртуальных выклікаў: кожны клас дадае свае палі пасля палёў бацькоўскага:

```
void PeripheralDevice::serialize(std::ostream& os) const
{
    Component::serialize(os);
    os << ';' << connection_;
}
void Mouse::serialize(std::ostream& os) const
{
    PeripheralDevice::serialize(os);
    os << ';' << dpi_;
}
```

4. РЭЗУЛЬТАТЫ РАБОТЫ І ТЭСЦІРАВАННЕ

Праграма была сабрана кампілятарам GCC са стандартам C++17 і ключамі `-Wall -Wextra -pedantic` без папярэджанняў. Праверка праводзілася на наборы файлаў з раздзела 2.1; у `mice.txt` дадаткова быў дададзены наўмысна пашкоджаны радок.

Табліца 4.1 — Тэставыя сцэнарыі і атрыманыя вынікі

№	Сцэнарый	Вынік
1	Загрузка з файлам, дзе ёсць радок «999;X;broken»	Радок прапушчаны, паказана паведамленне пра нестачу поля і нумар радка, астатнія дадзеныя загружаны
2	Пошук артыкула, якога няма ў каталогу	Выведзена паведамленне «Тавар з артыкулам … не знойдзены», праграма працягвае працу
3	Рэдагаванне кошту на значэнне −5	Валідацыя адхіліла значэнне, тавар не змяніўся, запіс у гісторыю не дададзены
4	Даданне, змяненне кошту, выдаленне, затым тры адмены і чацвёртая лішняя	Стан цалкам вернуты да пачатковага; чацвёртая адмена выдала паведамленне «Няма дзеянняў для адмены»
5	Захаванне пасля поўнага адкату	Файлы катэгорый па змесце супадаюць з зыходнымі
6	Сартаванне па кошце ўласным алгарытмам	Тавары ўнутры кожнай катэгорыі выведзены ў парадку ўзрастання кошту
7	Фарміраванне справаздачы па ўсіх 14 тавараў	У `inventory_report.txt` запісана 14 радкоў і агульны кошт 73527.50

5. КІРАЎНІЦТВА КАРЫСТАЛЬНІКА

Для збіркі патрэбны кампілятар з падтрымкай C++17 і CMake: каманды `cmake -S . -B build` і `cmake --build build`. Праграма запускаецца з указаннем каталога з дадзенымі: `shop data` (па змаўчанні выкарыстоўваецца каталог `data`). У гэтым каталогу павінны быць `categories.txt` і файлы катэгорый; `logs.txt` і `inventory_report.txt` ствараюцца аўтаматычна.

Пасля запуску выбіраецца роля: **1** — адміністратар, **2** — карыстальнік. Пункты меню:

- **1** — паказаць каталог (табліцы па катэгорыях);
- **2** — пошук (па артыкуле, вытворцы, назве, дыяпазоне кошту);
- **3** — сартаванне (па кошце, назве, колькасці; уласным алгарытмам або `std::sort`);
- **4** — стварыць справаздачу `inventory_report.txt`;
- **5–9** (толькі адміністратар) — даданне, змяненне, выдаленне, адмена апошняга дзеяння і захаванне ў файлы;
- **0** — выхад (адміністратару прапануецца захаваць змены).

Пры няправільным уводзе праграма паказвае прычыну памылкі і паўторна запытвае значэнне.

ЗАКЛЮЧЭННЕ

У ходзе курсавой працы спраектавана і рэалізавана на мове C++ інфармацыйная сістэма па продажы камп'ютарных камплектуючых. Распрацавана іерархія з трох узроўняў наследавання (`Component`, `PeripheralDevice` і `InternalHardware`, пяць канкрэтных класаў), створаны ўласныя шаблонны кантэйнер з ітэратарам і ўласныя алгарытмы, а таксама выкарыстаны кантэйнеры і алгарытмы STL. Захаванне дадзеных арганізавана ў тэкставых файлах па катэгорыях, пры загрузцы выконваецца праверка фармату.

Рэалізаваны даданне, рэдагаванне, выдаленне, пошук па некалькіх прыкметах, сартаванне, справаздача па рэштках, журналіраванне, абмежаванне правоў па ролях, іерархія ўласных выключэнняў і адмена дзеянняў па шаблоне «Каманда». Тэставанне пацвердзіла, што пашкоджаныя дадзеныя і недапушчальны ўвод не прыводзяць да аварыйнага завяршэння праграмы.

Магчымыя напрамкі развіцця: працяг гісторыі адмены паміж сеансамі, паўторнае выкананне адмененых дзеянняў (redo), графічны інтэрфейс, перанос дадзеных у базу дадзеных і падтрымка заказаў пакупнікоў.

СПІС ВЫКАРЫСТАНЫХ КРЫНІЦ

[1] Гаджинский, А. М. Логистика : учебник для бакалавров / А. М. Гаджинский. – 21-е изд. – Москва : Дашков и К, 2021. – 420 с.

[2] Радченко, М. Г. 1С:Предприятие 8.3. Практическое пособие разработчика. Примеры и типовые приемы / М. Г. Радченко, Е. Ю. Хрусталева. – Москва : 1С-Паблишинг, 2020. – 964 с.

[3] Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Бос. – 4-е изд. – Санкт-Петербург : Питер, 2021. – 1120 с.

[4] Луцик, Ю. А. Объектно-ориентированное программирование на языке С++ : учебное пособие / Ю. А. Луцик, В. Н. Комличенко. – Минск : БГУИР, 2008. – 266 с.

[5] Страуструп, Б. Язык программирования С++. Краткий курс / Б. Страуструп ; пер. с англ. – 2-е изд. – Москва : Вильямс, 2019. – 320 с.

[6] Дейтел, Х. Как программировать на С++ / Х. Дейтел, П. Дейтел ; пер. с англ. – 8-е изд. – Москва : Бином, 2021. – 1032 с.

[7] Буч, Г. Объектно-ориентированный анализ и проектирование с примерами приложений на С++ / Г. Буч. – 3-е изд. – Москва : Вильямс, 2020. – 496 с.

[8] Шилдт, Г. С++ : базовый курс / Г. Шилдт. – 3-е изд. – Москва ; Санкт-Петербург : Диалектика, 2019. – 620 с.

[9] Visual Studio Code Documentation : C++ programming with Visual Studio Code [Electronic resource]. – Mode of access : https://code.visualstudio.com/docs/languages/cpp. – Date of access : 20.09.2026.